

Solution Requirements

Date	28 June 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	No user login is required. The application is publicly accessible for flood prediction.	Low
2	Authorization levels	Single user access. No separate roles or permission levels are implemented.	Low
3	External interfaces	The system interacts with a trained Random Forest model (floods.save) through the Flask backend.	High

4	Transactions processing	Users enter weather parameters, the system validates the inputs, sends them to the ML model, and returns the flood prediction.	High
5	Reporting	Displays prediction results as Flood Chance or No Flood Chance on the result page.	Medium
6	Business rules, etc.	All input fields must contain valid numeric values. The model predicts flood risk based on trained weather data.	High
7	Compliance to laws or regulations	No personal or sensitive user data is collected or stored. Basic data privacy practices are followed.	Medium
8	Other	Users can perform multiple predictions using the Predict Again option without restarting the application.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Prediction results should be generated quickly after user input.	Response time less than 2 seconds
2	Scalability	The application should support multiple users making predictions simultaneously after deployment.	Supports 100+ concurrent users
3	Security & Data Privacy	Input validation is performed, and no user data is permanently stored.	Secure input validation and no data storage
4	Reliability & Availability	The application should remain available with minimal downtime.	99% uptime
5	Usability & Accessibility	The interface should be simple, responsive, and easy for users to understand.	Easy navigation with clear input fields and result display
6	Other	The system should be maintainable, allowing future updates such as live weather API integration and improved ML models.	Modular Flask architecture with reusable code